

Observability and Evaluation for Generative AI and Agentic Systems:

A Working Paper for a Deep-Dive Workshop

Workshop Working Paper
Observability and Evaluation Deep-Dive Workshop
October 6, 2026

Abstract—Generative artificial intelligence applications can return technically successful responses while producing unsupported answers, choosing unsuitable tools, or violating task policies. Conventional service monitoring therefore needs to be connected to explicit evaluation of application behavior. This working paper develops a practical framework for an observability and evaluation deep-dive workshop. It covers observability fundamentals, agent execution, OpenTelemetry, trace design, language-model telemetry, evaluation workflows, Arize observability, production monitoring, cost accounting, security and privacy, incident analysis, maturity assessment, and a hands-on session. The proposed architecture connects request traces, versioned evaluation records, operational metrics, and curated regression datasets through stable identifiers. It separates transport errors from semantic failures, treats missing evaluation evidence explicitly, and recommends release decisions based on paired experiments and risk-specific acceptance criteria. A reproducible synthetic exercise demonstrates how to investigate retrieval, tool, and authorization failures without presenting simulated outputs as model benchmark results. The paper is a technical synthesis and workshop design, rather than an empirical validation of a particular platform. Its main contribution is an actionable method for converting production evidence into measured improvements while preserving trace portability and controlled access to sensitive content.

Index Terms—AI agents, Arize, evaluation, generative AI, observability, OpenTelemetry, reliability, tracing.

I. OBSERVABILITY FUNDAMENTALS AND SCOPE

A. Problem and Workshop Objectives

An enterprise assistant may complete an HTTP request, consume a reasonable number of tokens, and still give the wrong policy advice. An agent may produce a plausible final answer after an unauthorized tool call. These examples make two questions distinct: did the service execute correctly, and did the application accomplish an acceptable task? The workshop treats operational reliability and behavioral quality as related measurements with different evidence requirements.

The established monitoring baseline includes latency, traffic, errors, and saturation, commonly called the four golden signals [1]. OpenTelemetry organizes telemetry into signals including traces, metrics, and logs [2]. This paper builds on those concepts and proposes an additional application-level evaluation contract. The contract identifies the task, available evidence, allowed actions, success criteria, and evaluation limitations before a score is interpreted.

Participants should leave able to design a trace that explains a failed task, distinguish a provider failure from an unsupported answer, join evaluations to the correct execution, define useful service-level indicators (SLIs), estimate cost per acceptable outcome, and turn an incident into a regression test. The intended audience includes platform engineers, AI application developers, reliability engineers, evaluators, and security reviewers. Familiarity with APIs and distributed applications is assumed; prior experience with an observability vendor is not required.

The paper follows the twelve subjects in the supplied workshop agenda. In the agenda, “Overwatch” refers to the Arize observability platform, as clarified by the workshop requester. The term is used here as workshop terminology; it is not asserted to be a separate official Arize product name. Section VII consequently discusses Arize AX and Phoenix using their documented names.

B. Monitoring, Observability, and Evaluation

Monitoring tracks declared indicators and conditions. Observability supports investigation using available execution evidence. Evaluation applies a defined criterion to an output, action, trajectory, or resulting state. A trace establishes what was recorded during execution; an evaluator determines how that evidence meets a rubric. Neither alone proves that every relevant event was captured or that the rubric represents all user needs.

Table I gives the proposed workshop interpretation of common evidence. The “limitation” column is as important as the measurement: a fast response is not a correctness label, a model explanation is not proof of its internal process, and a user rating is not an exhaustive safety assessment.

The proposed unit of analysis is an **eligible task execution**, identified by a root trace and an application run identifier. A task is operationally successful when it satisfies the declared execution contract, behaviorally acceptable when its evaluated outcome satisfies the quality and policy contract, and unknown when required evidence is missing. This three-state interpretation prevents unscored tasks from silently entering the quality-success numerator.

TABLE I
EVIDENCE AND ITS INTERPRETATION

Evidence	Principal question	Interpretation limit
Metrics	How often and how much?	Aggregation hides individual failures.
Traces	Which recorded operations occurred?	Missing spans leave gaps in the execution.
Structured logs	What event or decision was recorded?	Free text alone may lack stable joins.
Evaluations	Did evidence meet a stated criterion?	Scores depend on rubric and evaluator.
User and business outcomes	Was the result useful in context?	Feedback can be sparse and selective.

C. Research Questions and Evidence Boundaries

Three questions guide the working paper. RQ1 asks what instrumentation is sufficient to reconstruct model, retrieval, and tool execution. RQ2 asks how evaluation results can support diagnosis and release decisions without introducing hidden sampling or judging errors. RQ3 asks how organizations can operate this process with useful cost, privacy, and reliability controls.

The method is a synthesis of primary specifications, research papers, and product documentation, followed by a proposed architecture and workshop protocol. All numeric thresholds, timing allocations other than the agenda’s nine-minute closing session, maturity levels, and synthetic examples are workshop proposals. No production benchmark, customer deployment, or measured improvement is claimed. Source pages were checked on October 6, 2026. Living specifications and vendor workflows should be pinned and rechecked before a workshop deployment.

II. GENAI AND AGENT OBSERVABILITY

A. From a Model Call to a Task Execution

Language-model quality is multidimensional. HELM illustrates evaluation across scenarios and multiple metrics rather than a single accuracy measure [3]. Agentic systems add interactions with tools and the environment; ReAct is an influential example of interleaving language-model processing and actions [4]. Retrieval-augmented generation (RAG) combines retrieval with generation to supply external knowledge [5]. These systems motivate instrumentation at application boundaries, rather than treating every problem as a model-output problem.

For the workshop, a support assistant retrieves policy evidence, chooses an answer or an authorized read-only tool, validates the result, and returns a response or an explicit abstention. The root task span owns end-to-end timing and task outcome. Child spans represent retrieval, reranking, model invocation, tool execution, and output checks. Record the model and configuration actually used, including fallback attempts, rather than only the requested default.

The application should record externally visible decisions: selected tool, sanitized arguments or their schema, action

authorization, tool result category, retry count, stopping reason, and confirmed state change. Logging hidden chain-of-thought is not a prerequisite for useful observability. A generated rationale is an output artifact and may not accurately describe the process that produced an action. Prefer direct execution evidence and concise, application-generated decision records.

B. Outcome, Trajectory, and State

Agent evaluation must consider both the final outcome and relevant intermediate behavior. A correct answer can conceal an excessive or unauthorized action; a different tool sequence can still be valid. The tau-bench research evaluates tool-agent-user interaction against task goals and resulting database state, and examines consistency across repeated trials [6]. Its benchmark results do not establish the quality of the workshop application.

The proposed evaluation contract therefore specifies permitted tools, prerequisites for consequential actions, a target final state, and acceptable stopping conditions. Use a partial order for required dependencies: authorization must precede a protected action, and evidence retrieval must precede an evidence-grounded answer. Do not require an exact linear trajectory when legitimate alternative paths exist. Evaluate final-state invariants independently of the fluency of the final response.

For repeated runs, report the fraction of tasks that succeed in every tested repetition alongside the ordinary per-run success rate. Keep task identity and repetition identity separate. A stochastic application can appear strong on an average while repeatedly failing the same sensitive task. A few repetitions in a workshop demonstrate the concept but do not estimate production reliability precisely.

C. Failure Taxonomy

The workshop taxonomy distinguishes infrastructure failure, retrieval failure, generation failure, tool failure, policy failure, and evaluation failure. Examples include a provider timeout, a stale policy document, an unsupported answer despite adequate context, a tool response that violates its schema, a denied action executed anyway, and a judge that times out. Assigning the last category to the application would corrupt quality metrics.

Each incident should retain a primary observed failure and contributing factors. The primary category is not automatically a causal conclusion: a retrieval miss and a poor answer can co-occur without establishing which change caused the result. Controlled replay, configuration comparison, and independent outcome checks provide stronger evidence.

III. OPENTELEMETRY FOUNDATIONS

A. Instrumentation and Transport

OpenTelemetry provides instrumentation APIs and SDKs and a vendor-neutral Collector. The Collector receives, processes, and exports telemetry [7]. OTLP defines telemetry transport and encoding; its export responses and partial-success behavior matter when diagnosing ingestion loss [8].

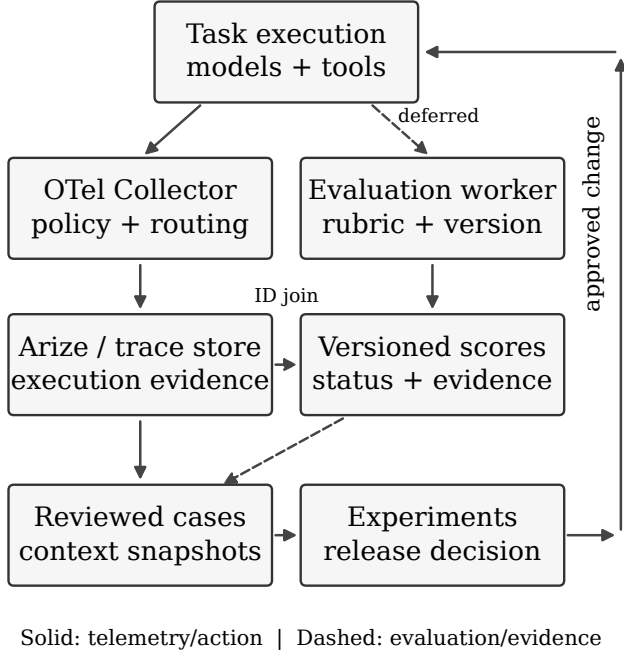


Fig. 1. Proposed reference architecture. Request telemetry and asynchronous evaluations share stable identifiers; curated failures feed versioned experiments and release decisions.

These components provide a collection and transport foundation. Application evaluation criteria and decision policies remain responsibilities of the system being observed.

The proposed architecture in Fig. 1 uses automatic instrumentation for supported network and framework boundaries, supplemented by manual spans for task orchestration, custom tools, authorization decisions, and semantic outcomes. Automatic instrumentation is useful but does not infer the application’s business success condition. A wrapper that sees an HTTP 200 cannot establish whether the returned advice was grounded in the right policy.

B. Collector Design

Collector configuration separates receivers, processors, exporters, and enabled service pipelines. Processor order controls transformation order, and merely declaring a component does not activate it [9]. A proposed organizational pipeline accepts OTLP, applies resource normalization and content controls, enforces bounded buffering, and routes approved telemetry to Arize and an operational backend where needed.

For this design, classify and minimize sensitive fields at instrumentation time, before any exporter or diagnostic logger can disclose them. Collector controls provide a second enforcement point. Protect the Collector itself: configure authentication for its actual exposure, constrain network access, monitor rejected data and export failures, and test the effect of an unavailable downstream destination.

A gateway can centralize policy and credentials, while local agents can reduce application coupling to backend details.

Choose the topology through measured throughput, failure behavior, and administrative constraints. Routing the same trace to two destinations improves investigative flexibility but increases storage and the number of places where access controls must be verified.

C. Resource Identity and Telemetry Health

Use resource identity to distinguish service, deployment environment, release, and region. The proposed workshop adds a schema revision and instrumentation revision to the run manifest. Keep application release, prompt version, retrieval snapshot, evaluator version, and vendor integration configuration separately identifiable. This supports comparison without pretending that a mutable model alias is an immutable model build.

Track telemetry coverage as an observable property. Reconcile eligible task counts from the application with ingested root spans, and reconcile model-call counts with captured model spans. Report propagation gaps, dropped spans, export latency, and evaluation join failures. A dashboard with no errors can indicate missing telemetry. Sampling must be accounted for before interpreting any reconciliation gap as loss.

IV. TRACE DESIGN AND STANDARDS

A. Span Boundaries and Context

The OpenTelemetry tracing API defines spans, span contexts, attributes, events, links, and status [10]. W3C Trace Context specifies interoperable propagation through `traceparent` and `tracestate` [11]. In the workshop design, the task boundary is a root or entry span; each timed operation receives a child span when it is causally nested within that execution.

For asynchronous work, preserve causal relationships explicitly. A worker may extract the incoming context to continue a task trace, or start a new trace and link it to a triggering span when that better represents its lifecycle. Do not assign a single fabricated parent to a batch with multiple independent triggers. A long conversation should use a separate session identifier across per-turn traces, rather than an indefinitely open root span.

Name operations consistently and keep user text out of span names. Put case-specific evidence into controlled attributes or references. Test propagation through queues, thread pools, retries, and fan-out. Similar timestamps or a shared user identifier do not establish parentage. Every retry is an attempt with an observable result; the logical task outcome is recorded separately.

B. Status and Semantic Outcomes

A span status communicates execution status; the paper’s task-quality fields communicate evaluated meaning. A model API invocation can execute without a transport error while the application’s answer fails a grounding rubric. Conversely, a temporary tool failure may be recovered and the task may succeed. Preserve both facts rather than overwriting every child status with the final task label.

The proposed schema uses `workshop.task.outcome` for application state and a separate evaluation record for rubric judgments. “Blocked by policy” is a useful outcome, not automatically an infrastructure error. Include timeout, canceled, refused, abstained, failed, completed, and unknown where the application contract distinguishes them. Record a documented stopping reason when an agent exhausts its step or time budget.

C. Baggage, Sampling, and Cardinality

Baggage can propagate application context across service boundaries, but it may be sent to downstream systems and does not automatically become span attributes [12]. The workshop permits only approved, minimal propagation values. Never use baggage for raw prompts, credentials, or unrestricted personal data. An opaque identifier can still be sensitive or linkable and needs access and retention controls.

Head sampling decides early, while tail sampling can select traces based on later evidence such as errors or latency [13]. A tail sampler cannot recover spans already discarded upstream. Retain a known-probability baseline sample for population estimates and a separate diagnostic sample for selected failures. Record which population a dashboard describes. If a quality label arrives after a tail sampler’s decision window, preserve an independently selected evaluation cohort or store the required evidence before that decision.

Do not place trace IDs, user IDs, prompts, or document IDs in ordinary metric labels. Use bounded dimensions such as model family, environment, task class, and error category; keep high-cardinality identifiers in trace or evaluation stores. Histogram aggregation should preserve the intended units and population. Averaging per-instance p95 values does not produce the global p95.

V. AI AND LLM-SPECIFIC TELEMETRY

A. Semantic Conventions and Versioning

OpenTelemetry’s GenAI conventions are maintained in a dedicated repository; the earlier website location explicitly directs readers there. The inspected GenAI client-span document is labeled Development [14]. OpenInference supplies an additional AI-oriented convention set used by Arize [15]. The workshop keeps these convention sets distinct: common OTLP transport does not make every attribute or operation taxonomy equivalent.

For example, GenAI spans use keys such as `gen_ai.request.model`, `gen_ai.response.model`, and `gen_ai.usage.input_tokens`; OpenInference includes `llm.model_name`, `llm.token_count.prompt`, and `openinference.span.kind`. These examples illustrate different representations, not a universal one-to-one conversion. Requested and returned model names, token categories, and operation kinds must retain their meaning when normalized [14], [15].

Pin an instrumentor version and a semantic-convention revision. Preserve original emitted attributes when adding a normalized view, document the transformation, and test representative traces after an upgrade. The GenAI client convention

TABLE II
PROPOSED MINIMUM APPLICATION EVIDENCE CONTRACT

Evidence group	Required content	Diagnostic purpose
Identity	Trace/span IDs; run and case IDs	Join execution and evaluation.
Configuration	Release, prompt, model, retrieval revisions	Compare reproducible configurations.
Model operation	Provider result, duration, usage provenance	Separate execution and resource use.
Retrieval	Corpus snapshot, top-k IDs, access outcome	Inspect missing or stale evidence.
Tool operation	Tool, attempt, sanitized schema, result	Inspect actions and recovery.
Policy	Authorization outcome and policy revision	Verify permitted behavior.
Content handling	Capture policy, redaction state, artifact reference	Explain accessible evidence.
Evaluation	Target IDs, rubric/revision, score/status	Interpret judgments and coverage.

groups automatic retries within a logical operation span [14]; preserve that span and capture individual attempts beneath it when needed. Do not silently emit conflicting duplicate token counts and then sum them. The workshop’s `workshop.*` fields below are proposed custom fields, not OpenTelemetry or Arize standards.

B. Streaming, Usage, and Evidence Capture

Measure request latency from the declared application boundary. For streaming, also measure time to first token or first meaningful output at a specified observation point, output-stream duration, and cancellation. Provider-reported latency, server-observed streaming latency, and user-visible latency have different boundaries. A connection keepalive should not count as a generated token. Do not label a post-hoc batch timestamp as time to first token.

Record token usage reported by the provider separately from local estimates. Preserve missing usage as missing, including requests that fail before usage is returned. Track cache and modality categories where available, and retain whether a number represents billed units or consumed units. Reasoning-token counts can be useful cost evidence without exposing hidden reasoning text. Token usage is not a quality metric.

The proposed capture policy offers metadata-only, redacted-content, and approved full-content modes. Default to the least content that answers the operational question. Keep protected payloads in an appropriately controlled artifact store when needed; a trace stores the reference and capture-policy revision. Hashes alone do not guarantee privacy, especially for short predictable text. Redaction can remove evidence needed by an evaluator, so the evaluation record must explain that limitation.

C. Evaluation Records as Independent Artifacts

Many evaluations execute after the application span has ended. Do not assume an SDK can mutate an already exported

span or extend it indefinitely. Store evaluations as linked records or annotations, and trace the evaluation worker itself when useful. A backend may expose annotations beside the original span without changing the span’s execution timing.

The proposed record key is (run_id, target_trace_id, target_span_id, evaluator_id, evaluator_version, evaluation_attempt). Each record includes task/case identity, dataset revision when applicable, a timestamp, rubric dimensions, score or label, status, concise evidence references, and judging configuration. Status values include scored, abstained, failed, and not-applicable. Versioned retries remain distinguishable; an idempotent write prevents an accidental duplicate from changing aggregate denominators.

VI. EVALUATION AND OBSERVABILITY CONVERGENCE

A. The Evidence-to-Improvement Loop

RAG evaluation needs to distinguish retrieval quality from generation quality. Ragas provides a research framework for evaluating dimensions such as retrieved context and grounded generation [16]. The workshop’s broader protocol adds task-specific deterministic checks, human review, and state-based agent checks. It does not treat a reference-free score as proof of factual correctness or regulatory compliance.

The proposed loop has six stages: define the task contract; capture approved execution evidence; run an appropriate evaluator; investigate failures; curate reviewed cases into a versioned dataset; and compare a candidate configuration with a baseline before rollout. The root trace or relevant child span joins each score to the evidence that was actually evaluated. A dataset should retain the context snapshot used during evaluation, rather than retrieving a different live corpus later and calling it the same test.

An evaluation is useful only when it can change a decision. A schema validator can block a malformed tool result. A grounding assessment can send an answer for review. A paired experiment can reject a prompt regression. A production sample can identify a new task class for the next dataset. Define these actions and their owners before building a large collection of score charts.

B. Evaluator Choice and Calibration

Use deterministic evaluators for exact formats, required fields, numeric constraints, allowed-tool checks, and verifiable final-state invariants. Use expert human review for ambiguous task quality, policy interpretation, and calibration. Model-based judges can scale some semantic judgments, but research documents position, verbosity, and self-enhancement biases and reasoning limitations [17]. Treat their outputs as measurements with error, not as ground truth.

The proposed calibration procedure uses a separately reviewed set covering ordinary, ambiguous, and adversarial examples. Reviewers first label independently, reconcile disagreements, and preserve unresolved cases. Compare a judge with the agreed labels by task slice using a confusion matrix and false-positive/false-negative rates. Judge agreement alone

TABLE III
PROPOSED EVALUATION RUBRIC AND EVIDENCE

Dimension	Measurement	Evidence requirement
Answer correctness	Task-specific pass/fail or expert rating	Trusted expected outcome.
Grounding	Supported/checkable claims	Accessible source passages.
Retrieval	Recall@k or reviewed relevance	Known relevant evidence set.
Tool validity	Schema and allowed-action checks	Tool arguments and result category.
Agent success	Required final-state invariants	Independently observed state.
Policy adherence	Authorization and prohibited-action checks	Policy revision and action audit.
User usefulness	Task completion or reviewed usefulness	User/task context and follow-up.

does not establish application correctness when the labels or rubric are poor.

Pin the rubric, judge model identifier, prompt/template revision, output parser, and inference settings. Blind configuration labels where possible, randomize presentation order in pairwise judgments, and check swapped-order consistency. Give a judge only the evidence required for its task. Delimit untrusted model outputs and retrieved text; a malicious passage can target the evaluator as well as the application. If the judge abstains or its parser fails, keep the result unknown and report coverage.

C. Metric Definitions and Missing Evidence

For a fixed task set, define operational availability and judged quality separately:

$$\begin{aligned} A &= N_{\text{operational success}} / N_{\text{eligible}} \\ Q &= N_{\text{acceptable}} / N_{\text{scored}}. \end{aligned} \quad (1)$$

Define evaluation coverage as $E = N_{\text{scored}} / N_{\text{eval}}$, where N_{eval} counts evaluation-eligible tasks. The denominators need not be identical: some tasks may legitimately be inapplicable to a specific rubric. Report the inapplicability rule, evaluator failures, and excluded tasks. An HTTP success code alone does not determine operational success if the contract requires a valid response body.

For retrieval, $R@k = |D_k \cap D_{\text{relevant}}| / |D_{\text{relevant}}|$, where D_k is the retrieved set and D_{relevant} is the known relevant set for that case. If no relevance labels exist, report reviewed relevance or another named proxy rather than calling it recall. A similarity score is not a calibrated probability of relevance.

For grounding, the proposed support ratio is $G = N_{\text{supported}} / N_{\text{checkable}}$. Also report how many claims were checkable and the extraction method. If there are no checkable claims, the ratio is not applicable; it is not automatically perfect. Grounding to a stale or incorrect source can still produce an incorrect answer. Evidence support and truth therefore remain separate dimensions.

D. Offline Experiments and Release Decisions

Build an initial dataset from representative tasks, reviewed incidents, edge cases, policy-sensitive actions, and known failure modes. Separate development and calibration examples from a frozen release holdout. Record the origin and intended population of each slice. Repeatedly tuning against the holdout eventually makes it development data; replace it under a documented process.

Compare baseline and candidate on the same cases, context snapshots, and task budgets. Let $s_{i,B}$ and $s_{i,C}$ be case-level scores. The paired mean difference is

$$\hat{\Delta} = \frac{1}{n} \sum_{i=1}^n (s_{i,C} - s_{i,B}). \quad (2)$$

Report a confidence interval and relevant slice results, not just this mean. For binary proportions, a Wilson interval avoids some weaknesses of a simple normal approximation [18]. For paired changes, the proposed analysis resamples paired cases; if multiple runs share a conversation, resample at the conversation or task-cluster level. Repeated generations of the same case are not independent new tasks.

Predeclare acceptance criteria before observing candidate results. A proposed gate requires all critical policy invariants to pass, no unacceptable quality regression on mandatory slices, adequate evaluation coverage, and latency and cost within the task budget. Passing a small test set does not prove zero residual risk. If evidence is insufficient, the decision is “more validation required,” not an invented pass.

E. Online Evaluation and Dataset Curation

Production evaluation uses a known baseline sample for population tracking plus a targeted cohort for rare or suspected failures. Do not merge these cohorts into an unweighted population estimate. Weighted estimation is appropriate only when inclusion probabilities and the sampling unit are known; some incident-selected data support diagnosis but not prevalence estimates.

Curate production failures before reusing them. Remove prohibited content, retain the needed policy and context snapshot, obtain a reviewed expected outcome, deduplicate correlated incidents, and record the original slice and selection method. Keep evaluator drift separate from application drift by rescoreing a stable calibration set after a judge change. Preserve both old and new evaluation versions to support an interpretable comparison.

VII. OVERWATCH: ARIZE AND THIRD-PARTY OBSERVABILITY PLATFORMS

A. Arize Instrumentation and Evaluation

Arize AX documents ingestion of OpenTelemetry spans and interpretation of OpenInference attributes for AI-specific visualization [19]. Its documented online evaluation workflow attaches evaluation results to the selected production evidence, while offline experiments compare task outputs and evaluation results on datasets [20], [21]. Arize also documents

TABLE IV
BACKEND ROLES AND WORKSHOP ACCEPTANCE CHECKS

Backend	Documented foundation	Proposed proof to perform
Arize AX	OTel/OpenInference; production and experiment evaluations [19]–[22]	Join a failed evaluation to the correct trace.
Phoenix	Open-source AI tracing and evaluation [24]	Inspect synthetic runs and export reviewed cases.
Grafana Tempo	Distributed tracing and log/metric correlation [25]	Follow a service metric to a task trace.
Langfuse	OTel integration for LLM observability [26]	task relationships on ingestion.

a trajectory-evaluation workflow that groups tool calls and associates a trace-level result with the root span [22]. These are documented capabilities, not benchmark findings from this paper.

The proposed Arize workshop workflow begins with a separate test project and synthetic evidence. Instrument one task, confirm the root and children, inspect model and tool metadata, and verify that content-capture policy is respected. Run deterministic checks first, then a calibrated semantic evaluator if one is available. Filter failed results by configuration and task class, inspect the linked trace, and export a reviewed failure into a regression dataset. Compare a baseline and a candidate against that dataset, retaining run and evaluator revisions.

Project/environment separation, authentication, deployment region, retention, and permitted content should be established before sending real user data. Use the transport and authentication configuration specified for the actual deployment; Arize documents transport-specific authentication settings [23]. Exporting a span does not prove it arrived in the intended project. Confirm ingestion with a distinctive synthetic case and known trace identifier.

B. Phoenix and Other Backend Roles

Phoenix is Arize’s open-source platform for AI observability and evaluation, built on OpenTelemetry and OpenInference; Arize AX is the managed enterprise platform described alongside it [24]. Phoenix offers a useful workshop environment when local experimentation is preferred. The exercise does not assume identical APIs, operational controls, or commercial availability across Phoenix and AX.

Conventional tracing and AI evaluation backends can serve complementary roles. Grafana Tempo documents distributed tracing, links with logs and metrics, and support for OpenTelemetry [25]. Langfuse documents an OpenTelemetry integration for LLM observability [26]. Table IV describes documented foundations and proposed acceptance checks, without ranking products or inferring equivalent evaluation semantics.

The selection protocol tests schema preservation, causal links, evaluation versioning, export/replay capability, content controls, ingestion loss visibility, and operating cost using

the same approved examples. A successful OTLP connection demonstrates transport interoperability; it does not establish that every backend interprets AI attributes or annotations identically. Retain a vendor-independent run manifest and dataset export so investigations are not dependent on a screenshot or an unrecoverable UI state.

VIII. PRODUCTION MONITORING

A. SLIs, SLOs, and Alerting

Service-level objectives (SLOs) turn indicators into explicit reliability targets. Google SRE describes error-budget burn-rate alerting and the use of multiple windows to balance responsiveness and noise [27]. The workshop applies that approach first to objective execution indicators; semantic-quality indicators need additional care because scores can be delayed, sampled, or unavailable.

For a chosen objective S , define burn rate as

$$B = \frac{N_{\text{bad}}/N_{\text{eligible}}}{1 - S}. \quad (3)$$

As an illustration, a 99% objective and a measured 2% bad-event rate produce a burn rate of 2. This calculation describes consumption relative to a budget; it does not specify a suitable alert threshold by itself. Choose window lengths and thresholds from the service’s traffic, response capability, and allowable budget consumption.

Maintain separate operational, latency, semantic-quality, policy, and telemetry-health indicators. For online quality, display evaluation lag, coverage, and the population sampled. Avoid urgent automated rollback based only on a small set of delayed, noisy judge labels. A verified prohibited state change may justify immediate action under the incident policy even when aggregate quality appears healthy.

B. Dashboards and Drift

The proposed operational dashboard shows eligible task traffic, operational outcomes, p50/p95/p99 request latency, streaming latency where applicable, retries, tool failures, model/provider breakdown, and saturation. The quality dashboard adds correctness, grounding, abstention, final-state success, evaluator status, and slice coverage. The cost dashboard relates usage and estimated spend to acceptable outcomes. Each dashboard links to representative evidence and states its sampling rule.

Compare prompt revisions, retrieval snapshots, model identifiers, task classes, languages, and approved coarse user cohorts. A change in input distribution is a signal to investigate, not proof of lower quality. Compare the same task slices or reweight a known sample before attributing a score shift to the model. A higher abstention rate may be a desirable policy improvement or a usefulness regression depending on the task contract.

C. Rollout and Recovery

Use offline gates, a limited rollout, and observed production outcomes together. Shadow execution can compare outputs but cannot fully validate state-changing tools unless actions are sandboxed or represented by a faithful dry-run model. Record the baseline, candidate, rollout fraction, observation period, and rollback condition. Preserve the configuration needed to revert and the evidence needed to explain the decision.

Instrument the telemetry and evaluation services themselves. A failed evaluation queue, expired export credential, or broken attribute transform can produce misleading quality trends. When the evidence pipeline fails, the appropriate status is reduced visibility or unknown quality. Define how long a rollout may continue with that condition and who makes the decision.

IX. OBSERVABILITY USES

A. Cost and Token Observability

The proposed request-level cost model uses mutually exclusive billable categories:

$$C_j = \sum_{a \in \mathcal{A}_j} \sum_{b \in \mathcal{B}_a} \frac{N_{ab} r_{ab}}{10^6} + C_{j,\text{other}}. \quad (4)$$

Here, a is a model-call attempt, b is a billable category, N_{ab} is its usage, r_{ab} is a versioned price per million units, and $C_{j,\text{other}}$ includes tool, retrieval, and evaluation costs where attributable. Category definitions must follow the provider’s billing rules. Cached input or reasoning units must not be charged twice when already included in another reported category. This is an accounting framework; no live vendor prices are assumed.

Attach currency, effective date, tariff revision, region/tier where relevant, and whether usage is reported or estimated. Include failed and retried calls when they incur charges. Distinguish model-usage estimates from invoice totals, which can include commitments, discounts, minimums, taxes, and non-request charges. Reconcile aggregate estimates periodically rather than treating a span field as an invoice.

The proposed **cost per acceptable task** is total attributable cost for the measured cohort divided by the number of tasks independently judged acceptable. Keep the numerator’s cohort aligned with the outcome population; an evaluated 10% sample cannot be divided directly into cost for all traffic. Report coverage and the allocation method. If no task is acceptable, the ratio is undefined, and report cost and failures separately.

Investigate long prompts, repeated retrieval, retry storms, unbounded agent steps, ineffective caches, and expensive judging before selecting an optimization. Test a cheaper model, shorter context, or cache change against the same quality and policy gates. Set time, step, and cost budgets at the task boundary; record budget exhaustion as an explicit stopping outcome.

B. Security, Privacy, and Compliance Observability

NIST’s Generative AI Profile identifies risks and suggested risk-management actions, including data privacy and information-security concerns [28]. OWASP describes prompt injection and sensitive-information disclosure as distinct application risks [29], [30]. Observability can provide evidence for investigating those risks; a trace or evaluator pass does not by itself demonstrate legal compliance or eliminate the vulnerability.

The proposed minimum controls are least-privilege tool access, authorization enforced outside the language model, tenant isolation, approved content capture, access auditing, and explicit retention. Retrieved text, model output, and tool results remain untrusted inputs to both applications and judges. Log authorization outcomes, policy revision, tool identity, redaction results, and state-change receipts. Keep secrets and raw authentication tokens out of telemetry.

Privacy controls apply to traces, logs, evaluation exports, datasets, screenshots, backups, and replay artifacts. Test metadata-only and redacted modes with synthetic identifiers; verify that an export or debug logger does not bypass them. Use opaque references where possible, but recognize that linkable identifiers can still expose sensitive relationships. Do not turn a production incident into a broadly shared dataset without reviewing its content and intended use.

For protected actions, an audit trail should establish who or what initiated the request, the approved identity or role context, the authorization decision, the action performed, and the resulting state. Keep that security evidence independent of ordinary diagnostic trace sampling. Retention and access policy should fit the organization’s actual obligations and deployment jurisdiction; the workshop does not prescribe a universal duration.

Measure control performance with stated denominators: prohibited actions prevented per attempted prohibited action, capture-policy violations per checked export, and access-control test outcomes per defined scenario. A high policy-block rate can reflect an attack, a changed workload, or an overly restrictive policy. Preserve those interpretations instead of treating the rate alone as success or failure.

C. Incident Management and Root Cause Analysis

Begin incident triage with user impact, affected task classes, time window, release and retrieval revisions, and operational symptoms. Locate representative failed executions and suitable comparison cases. Classify the observed failure, examine the recorded dependencies, and identify an evidence-backed hypothesis. Correlation with a deployment is a useful lead but does not establish causation.

In the workshop scenario, answers about policy limits fail after a retrieval change. The transport spans remain healthy, but the retrieved document snapshot is older than the expected policy. A grounded answer can therefore be supported by its context and still be wrong. Compare corpus revisions, reproduce the case against the prior snapshot, and independently

TABLE V
PROPOSED MATURITY EVIDENCE

Dimension	Level 2: correlated	Level 4: continuously improved
Execution coverage	Task and dependency counts reconcile.	Gaps trigger owned corrective work.
Trace/schema quality	Context and configuration joins work.	Upgrades have migration checks.
Evaluation quality	Versioned scores join to evidence.	Calibration and drift are reviewed.
Production reliability	SLO views link to failed executions.	Rollout and recovery are exercised.
Privacy/security	Capture policy and audit evidence join.	Controls are routinely tested.
Improvement workflow	Reviewed incidents become cases.	Paired gates guide measured changes.

check the expected policy outcome before naming the retrieval update as the cause.

Containment may include reverting a corpus or prompt revision, disabling a tool, enforcing a validated fallback, or increasing human review under the incident plan. Preserve approved evidence and avoid uncontrolled replay of state-changing calls. Record an incident timeline, confidence in the diagnosis, corrective action, owner, and validation outcome. A reviewed failure case then becomes a regression test with the relevant context and policy revision.

X. OBSERVABILITY MATURITY ASSESSMENT

A. Proposed Evidence-Based Rubric

The workshop proposes five maturity levels: 0, absent; 1, recorded; 2, correlated; 3, controlled; and 4, continuously improved. These are workshop levels, not an industry certification. Assess six dimensions independently: execution coverage, trace/schema quality, evaluation quality, production reliability, privacy/security evidence, and improvement workflow.

For each dimension, assign the highest level supported by demonstrated evidence, with prerequisites satisfied. A dashboard screenshot is insufficient evidence of controlled operations. Require a trace, a joined evaluation, a versioned dataset, a privacy test, or a completed incident-to-regression exercise as appropriate. Table V gives examples of what progression means.

B. Scoring and Improvement Priorities

An optional workshop summary is $M = \sum_d w_d m_d / (4 \sum_d w_d)$, with nonnegative declared weights and scores $m_d \in \{0, 1, 2, 3, 4\}$. Equal weights are the initial proposal. Report the full six-dimensional profile alongside the summary; the scalar is not validated and can hide a critical weakness. Missing authorization evidence cannot be compensated for by excellent cost dashboards.

Choose the next improvement from a demonstrated gap and an operational need. For example, if quality scores exist but do not identify the evaluated trace, prioritize stable joining

TABLE VI
PROPOSED FOUR-HOUR WORKING AGENDA

Agenda subject	Minutes	Expected output
1. Observability fundamentals	15	Task and success contract.
2. GenAI and agent observability	15	Failure taxonomy.
3. OpenTelemetry foundations	20	Telemetry pipeline design.
4. Trace design and standards	20	Root/child/link decisions.
5. AI/LLM telemetry	20	Minimum evidence schema.
6. Evaluation convergence	30	Rubric and release gate.
7. Arize and other platforms	15	Ingestion and join proof.
8. Production monitoring	20	SLI and alert design.
9. Observability uses	25	Cost, control, and incident view.
10. Maturity assessment	15	Evidence-backed gap profile.
11. Hands-on session	36	Trace, score, and regression case.
12. Commitment and Q&A	9	Owner and measurable next action.

before adding new metrics. If content controls are untested, validate those controls before expanding capture. Assign an owner, a deliverable, an acceptance test, and a review date to each selected improvement.

XI. HANDS-ON SESSION AND WORKSHOP PROTOCOL

A. Proposed Agenda and Prerequisites

Table VI allocates 240 minutes of working time across the supplied agenda, including the specified nine-minute commitment and Q&A segment. Breaks are additional. The 36-minute practical session assumes the facilitator has prepared the environment and participants have completed setup. A longer implementation clinic can follow when participants need to integrate real applications.

Prerequisites are a prepared Python environment with the companion archive’s pinned dependencies, a synthetic fixture set, the companion lab source, and an approved destination if live export is desired. Console/file output works without an Arize account or a model-provider key. Official Python instrumentation documentation explains tracer setup and span creation [31]. Arize/Phoenix setup is performed separately against the deployment’s current documentation.

B. Synthetic Task and Instrumentation

The companion exercise uses twelve synthetic policy-support cases with explicit expected answers and allowed actions. A fixed local policy snapshot and deterministic simulated generation make the expected outcomes inspectable. The baseline contains four intentionally inserted defects: stale retrieval, a missing-evidence answer, an unauthorized tool action, and a failed tool attempt. The candidate applies defined fixes. These data are intentionally constructed for teaching and cannot support a claim about a real model or platform.

Every simulated task emits a root execution and child retrieval, model-simulation, authorization, and tool spans as applicable. An evaluation-worker span links to the completed task. Each result carries trace/span IDs, case identity, application mode, evaluator version, score, and evidence category. All

token counts and costs in the simulation are absent or explicitly synthetic; do not mix them into real usage dashboards.

The abbreviated pattern below shows the link between completed execution and an asynchronous evaluation. `evaluate_case` is the task-specific deterministic function defined in the companion lab, and tracer-provider/exporter setup is deliberately outside the excerpt.

```
from opentelemetry import trace
from opentelemetry.trace import Link

tracer = trace.get_tracer("workshop.lab")
with tracer.start_as_current_span("task") as root:
    root.set_attribute("workshop.case.id", case.id)
    output = execute_case(case)
    origin = root.get_span_context()

with tracer.start_as_current_span(
    "evaluate", links=[Link(origin)]
) as evaluation:
    result = evaluate_case(case, output)
    write_record(origin, result)
```

OpenInference kinds and approved metadata can additionally make these spans easier to inspect in Arize. Label the model operation as simulation and preserve the distinction between the synthetic application and a real provider integration. The local JSON records are a workshop artifact, not a claimed Arize annotation-import format; use the platform’s documented workflow to attach results.

C. Practical Sequence and Acceptance Evidence

Minutes 0–6: run the baseline, open one root trace, confirm child relationships, and match one evaluation record to its target execution. Minutes 6–12: inspect the four injected defects and distinguish execution failure, semantic failure, and policy failure. Minutes 12–22: apply the deterministic rubric, review missing evidence, and inspect the separate evaluation-worker trace or linked record.

Minutes 22–30: run the candidate on the same fixtures, compare case-level outcomes, and explain which controlled change addresses each defect. Minutes 30–36: create one reviewed regression case, draft an incident note, and record a maturity gap. If a model-based judge is added, keep its judgments separate and compare them with the fixture oracle rather than silently replacing that oracle.

Successful completion requires an identifiable task trace, a justified outcome for each injected defect, an evaluation join with no orphan targets, one reproduced before/after case, and one documented next action. The exercise demonstrates instrumentation and diagnosis. Twelve deliberately designed cases, deterministic outputs, and a manually constructed candidate are insufficient for a production release recommendation.

D. Suggested Production Validation Extension

After the workshop, replace simulated generation with an approved application integration and build a larger reviewed task set. Predeclare the failure taxonomy, sampling population, task slices, acceptance thresholds, and primary metric. Compare baseline and candidate on paired cases, repeat stochastic

tasks, review evaluator disagreements, and measure telemetry overhead and ingestion completeness. Include content-control and authorization tests.

Publish the dataset specification, configuration manifest, run counts, missing-data accounting, and confidence intervals with any results. Test the proposed hypothesis that joined trace/evaluation evidence reduces diagnosis effort using comparable incident tasks and a defined timing procedure. Until such measurements exist, reduced debugging time and improved quality remain intended benefits of the framework, not demonstrated outcomes.

XII. COMMITMENT AND Q&A

A. Nine-Minute Closing Protocol

Use the first three minutes to identify one observed gap and one system where it matters. Use the next three minutes for a concrete commitment with an owner, deliverable, acceptance evidence, and review date. Use the final three minutes for unresolved questions that could change implementation. Participants should avoid commitments such as “improve observability” that have no observable completion condition.

Discussion prompts include: Which user outcome is currently unmeasured? Which action needs independent authorization evidence? Can every score identify its evaluated execution? What percentage of tasks is unknown because evaluation failed or evidence was unavailable? Which telemetry field creates more exposure than investigative value? Which incident should become the first reviewed regression case?

B. Proposed Thirty-Day Follow-Through

In days 1–7, the application and platform owners instrument one important task path and prove propagation and capture policy using approved fixtures. In days 8–14, the evaluation owner builds a reviewed dataset, calibrates the rubric, and verifies score-to-trace joins. In days 15–30, reliability and security owners exercise an alert, a rollback, and an authorization scenario; the team then uses a paired experiment to decide one scoped change.

Completion evidence is a versioned task contract, a complete example trace, an evaluation record with provenance, a reviewed regression dataset, a declared release gate, and an owned runbook. The maturity assessment is repeated using those artifacts. This follow-through is a proposed implementation path and should be adjusted to the organization’s risk, workload, and staffing.

XIII. CONCLUSION

Observability explains recorded execution; evaluation determines whether evidence satisfies a task-specific criterion. Generative AI and agent applications need both. The proposed framework joins traces, configuration history, quality judgments, cost accounting, and operational outcomes while treating missing evidence and evaluator failure explicitly. OpenTelemetry supports portable collection and context, while Arize provides documented AI observability and evaluation workflows within that foundation.

The working paper’s practical contribution is a shared evidence contract and an incident-to-regression process that participants can test during the workshop. Its architecture, maturity rubric, thresholds, and exercise are proposals rather than validated standards or empirical findings. The next step is controlled validation on representative workloads, including task-level uncertainty, security controls, telemetry overhead, and the usefulness of joined evidence for real incident diagnosis.

REFERENCES

- [1] R. Ewaschuk, “Monitoring distributed systems,” in *Site Reliability Engineering*, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. Sebastopol, CA, USA: O’Reilly Media, 2016. [Online]. Available: <https://sre.google/sre-book/monitoring-distributed-systems/>
- [2] OpenTelemetry Authors, “Signals,” OpenTelemetry documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/>
- [3] P. Liang *et al.*, “Holistic evaluation of language models,” arXiv:2211.09110, 2022, revised 2023, doi: 10.48550/arXiv.2211.09110. [Online]. Available: <https://arxiv.org/abs/2211.09110>
- [4] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” arXiv:2210.03629, 2022, revised 2023, doi: 10.48550/arXiv.2210.03629. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [5] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” arXiv:2005.11401, 2020, doi: 10.48550/arXiv.2005.11401. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [6] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “Tau-bench: A benchmark for tool-agent-user interaction in real-world domains,” arXiv:2406.12045, 2024, doi: 10.48550/arXiv.2406.12045. [Online]. Available: <https://arxiv.org/abs/2406.12045>
- [7] OpenTelemetry Authors, “Collector,” OpenTelemetry documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://opentelemetry.io/docs/collector/>
- [8] OpenTelemetry Authors, “OTLP specification,” OpenTelemetry specification. Accessed: Oct. 6, 2026. [Online]. Available: <https://opentelemetry.io/docs/specs/otlp/>
- [9] OpenTelemetry Authors, “Configuration,” OpenTelemetry Collector documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://opentelemetry.io/docs/collector/configuration/>
- [10] OpenTelemetry Authors, “Tracing API,” OpenTelemetry specification. Accessed: Oct. 6, 2026. [Online]. Available: <https://opentelemetry.io/docs/specs/otel/trace/api/>
- [11] W3C, “Trace Context,” W3C Recommendation, Nov. 23, 2021. [Online]. Available: <https://www.w3.org/TR/trace-context-1/>
- [12] OpenTelemetry Authors, “Baggage,” OpenTelemetry documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/baggage/>
- [13] OpenTelemetry Authors, “Sampling,” OpenTelemetry documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/sampling/>
- [14] OpenTelemetry Authors, “Semantic conventions for generative client AI spans,” official specification repository. Accessed: Oct. 6, 2026. [Online]. Available: <https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-spans.md>. Migration notice: <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
- [15] Arize AI and OpenInference Contributors, “OpenInference semantic conventions,” official specification repository. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/Arize-ai/openinference/blob/main/spec/semantic_conventions.md
- [16] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” arXiv:2309.15217, 2023, revised 2025, doi: 10.48550/arXiv.2309.15217. [Online]. Available: <https://arxiv.org/abs/2309.15217>
- [17] L. Zheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” arXiv:2306.05685, 2023, doi: 10.48550/arXiv.2306.05685. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [18] NIST/SEMATECH, “Confidence intervals,” in *e-Handbook of Statistical Methods*, sec. 7.2.4.1. Accessed: Oct. 6, 2026. [Online]. Available: <https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm>

- [19] Arize AI, “Tracing concepts,” Arize AX documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://arize.com/docs/ax/instrument/what-are-traces>
- [20] Arize AI, “Run online evals on traces,” Arize AX documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://arize.com/docs/ax/evaluate/run-evals-on-traces>
- [21] Arize AI, “Run offline evals on experiments,” Arize AX documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://arize.com/docs/ax/evaluate/run-evals-on-experiments>
- [22] Arize AI, “Agent trajectory evaluations,” Arize AX documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://arize.com/docs/ax/evaluate/evaluators/trace-and-session-evals/trace-level-evaluations/agent-trajectory-evaluations>
- [23] Arize AI, “Manual instrumentation,” Arize AX documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://arize.com/docs/ax/instrument/manual-instrumentation/>
- [24] Arize AI, “What is Arize Phoenix?,” Phoenix documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://arize.com/docs/phoenix/>
- [25] Grafana Labs, “Grafana Tempo,” Grafana Tempo documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://grafana.com/docs/tempo/latest/>
- [26] Langfuse, “OpenTelemetry integration,” Langfuse documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://langfuse.com/integrations/native/opentelemetry>
- [27] S. Thurgood *et al.*, “Alerting on SLOs,” in *The Site Reliability Workbook*, B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, and S. Thorne, Eds. Sebastopol, CA, USA: O’Reilly Media, 2018. [Online]. Available: <https://sre.google/workbook/alerting-on-slos/>
- [28] NIST, *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*, NIST AI 600-1, Jul. 2024, doi: 10.6028/NIST.AI.600-1. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
- [29] OWASP GenAI Security Project, “LLM01:2025 Prompt injection,” 2025 edition. Accessed: Oct. 6, 2026. [Online]. Available: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- [30] OWASP GenAI Security Project, “LLM02:2025 Sensitive information disclosure,” 2025 edition. Accessed: Oct. 6, 2026. [Online]. Available: <https://genai.owasp.org/llmrisk/llm022025-sensitive-information-disclosure/>
- [31] OpenTelemetry Authors, “Instrumentation,” OpenTelemetry Python documentation. Accessed: Oct. 6, 2026. [Online]. Available: <https://opentelemetry.io/docs/languages/python/instrumentation/>